

Chapter 2

Playwright Architecture – Explained Through Real Problems



Browser Engine

Playwright (Interface)



BrowserType (Factory)

– Chromium / Firefox / WebKit –



Browser (Process)



BrowserContext (User Session)



Page (Behavior)



Locator

(Element Finder)



iFrame

(Nested Page)

• Abstraction • Encapsulation • Isolation by Design



Chapter Intent

In ***Chapter 1***, we understood ***why modern automation struggles***.

In this chapter, we answer a more important question:

Why does Playwright feel more stable, predictable, and modern than traditional tools?

The answer is not features.

It is architecture and object-oriented design.



1. A Familiar Morning — But Something Feels Different

It's **10:05 AM**.

You rerun the same login test that failed yesterday.

- Same application.
- Same environment.
- Same data.

This time, it passes.

1. You didn't add a wait.
2. You didn't add retries.
3. You didn't touch the test logic.

Instead of asking what changed,

you start asking how this tool thinks.

That's where this chapter begins.



2. Automation Tools Are Systems, Not Scripts

Most engineers start automation by learning commands:

- Click
- Type
- Navigate
- Assert

But mature automation frameworks are not collections of commands.

They are systems made of objects, each with:

- A responsibility
- A lifecycle
- A boundary

Playwright is designed this way. ***To understand it, we must understand its objects.***

3. Browser Engine — The Foundation Everything Runs On

Definition

A Browser Engine is the core software responsible for:

- Rendering HTML and CSS
- Executing JavaScript
- Managing DOM, events, and network behavior

Common Engines

- Chromium → Chrome, Edge
- WebKit → Safari
- Gecko → Firefox

Why This Matters

Different engines behave differently.

Playwright talks *directly to these engines*, not through an external driver layer.

This is why Playwright tests are faster and more reliable.



4. Playwright — The Entry Point

`Playwright playwright = Playwright.create();`

Definition

Playwright is the root interface of the framework.

It:

- Initializes communication with browser engines
- Exposes capabilities (launching, context creation, tooling)
- Does NOT represent a browser itself

OOP Principle

Abstraction

You interact with what Playwright can do,
not how it communicates internally.

5. BrowserType — Choosing an Engine, Not a Browser

```
BrowserType chromium = playwright.chromium();
```

Definition

A BrowserType represents a specific browser engine configuration.

Examples:

- chromium
- firefox
- webkit

Responsibility

- Acts as a *factory*
- Knows how to launch browsers for that engine

OOP Principle

Factory Pattern + Polymorphism

Same API.

Different implementations.



6. Browser — A Real Process

```
Browser browser = chromium.launch();
```

Definition

A Browser represents a running browser process.

What It Manages

- Launching and closing the browser
- Connection to the engine
- Shared process-level resources

What It Is NOT

✗ Not a user ✗ Not a test ✗ Not a session

OOP Principle : Encapsulation

You don't manage OS-level browser processes — the object does.



7. BrowserContext — Where Stability Begins

```
BrowserContext context = browser.newContext();
```

Definition

A BrowserContext represents a single, isolated user session.

Each context has:

- Independent cookies
- Independent cache
- Independent storage
- Separate permissions

Real-World Mapping

👉 One BrowserContext = One User

What It Is NOT

✗ Not environment (QA / Prod)

✗ Not browser

Why This Matters

Parallel execution becomes safe because sessions never collide.

OOP Principle :

Single Responsibility + Isolation by Design

8. Page — Where Behavior Happens

```
Page page = context.newPage();
```

Definition

A *Page* represents a single browser tab.

Responsibilities

- Navigate to URLs
- Interact with UI
- Observe browser state
- Trigger auto-waits

Key Insight

A Page does not blindly execute actions.

It waits for readiness before acting.

OOP Principle : *Behavior Encapsulation*

9. Auto-Waits — Design, Not Magic

```
// When you write:
```

```
page.click("#login");
```

Playwright:

1. Waits for element existence
2. Waits for visibility
3. Waits for stability
4. Performs the action
5. Waits for navigation if needed

Why This Is Important

You are no longer responsible for timing.

You express *intent*.

OOP Principle : *Abstraction*

10. Locator — Intent Over Snapshot

```
Locator loginButton = page.locator("#login");
```

Definition

A *Locator* is a lazy reference to DOM elements.

The Real Problem

- Elements go stale
- DOM changes invalidate references
- Tests break due to re-rendering

Key Characteristics

- Evaluated at action time
- Automatically retries
- Avoids stale element issues
- **References should be resilient, not static.**

Why It Matters

Locators represent **what you want**, not what exists right now.



11. iFrame — A Page Within a Page

Definition

An *iFrame* is a nested browsing context inside a page.

It has:

- Its own DOM
- Its own JS execution
- Its own isolation

Playwright Advantage

iFrames are modeled explicitly, not implicitly — avoiding hidden failures.

OOP Principle : *Composition*

12. How Everything Fits Together

Browser Engine



Playwright (Interface)



BrowserType (Factory)



Browser (Process)



BrowserContext (User Session)



Page (Behavior)



Locator / iFrame

Each layer:

- Owns one responsibility
- Manages its lifecycle

This is **clean object-oriented design**.

Playwright's architecture is not layered for complexity.

It is layered to **separate concerns**:

- Engines → *browser behavior*
- Browser → *process lifecycle*
- Context → *user isolation*
- Page → *behavior and timing*
- Locator → *intent and resilience*

Each layer exists because a problem existed first.

13. Why This Architecture Reduces Flakiness

Flakiness reduces because:

- Timing is abstracted
- Isolation is guaranteed
- Responsibilities are clear
- Debugging is integrated

You stop writing defensive automation.

You start writing **behavior-focused automation**.



14. End of the Day — A New Trust

You didn't fight the browser today.

You worked with it.

And that's when you realize:

Stability isn't enforced with waits.

It's designed into the architecture.



Chapter 2 Key Takeaways

- Playwright is object-driven, not command-driven
- Interfaces hide complexity
- Contexts enable true isolation
- Pages observe before acting
- Good design eliminates flakiness

Reflection Questions

1. Which Playwright object solves your biggest automation pain today?
2. Where does your current framework break abstraction?
3. How much code exists only to handle timing?
4. Do your tests model real users accurately?
5. What would change if isolation was guaranteed by design?



Closing Thought

Great automation doesn't come from writing more code.

It comes from understanding ***what each object represents in the real world.***

Once the objects are clear,
the framework becomes obvious.



What Comes Next

Chapter 3 : Setting Up Playwright with Java — From Zero (Hands-On)

You're no longer just learning Playwright.

You're learning how modern automation frameworks are engineered.